

Video Factory — Runbook

Everything needed to go from zero to a finished video,
and to make each
subsequent video cheaper than the last.

PART 1 — Setup (one time)

Step 1. Wake the Supabase project

Your project `FarrukhGulomov's Project` is currently **INACTIVE** (paused).

Supabase pauses free-tier projects after a week of inactivity.

1. Open the Supabase dashboard
2. Select the project
3. Press **Restore project** — takes 2–5 minutes

Nothing below works until this is done.

Step 2. Apply the schema

Open **SQL Editor** in the dashboard, paste the contents of `01-schema.sql`,
and run it. Then verify:

```
select table_name from
information_schema.tables
where table_schema = 'public';
```

You should see: projects, characters, locations, templates, scenes, assets, generations.

Step 3. Get a fal.ai key

1. Sign up at fal.ai
2. Dashboard → API Keys → create a key
3. Add credit — start with \$10, not more. Enough for roughly 60–80 Lite-tier motion tests, which is more iteration than a first video needs.

Step 4. Connect fal.ai over MCP

Claude Desktop config

(macOS: ~/Library/Application

Support/Claude/claude_desktop_config.json,

Windows:

%APPDATA%\Claude\claude_desktop_config.json):

```
{
  "mcpServers": {
    "fal-ai": {
      "command": "npx",
      "args": ["-y", "fal-ai-mcp"],
```

```
    "env": {  
      "FAL_KEY": "your-fal-api-key"  
    }  
  }  
}
```

Restart Claude. Verify by asking it to list available fal models.

Step 5. Seed your locks

Insert what you already have, so you never retype it:

```
insert into characters (name, lock_text,  
notes) values  
(  
'Me', '<your fixed physical description –  
build, hair, face, wardrobe>',  
'Likeness breaks on: wide framing, hard  
low-key light, 3/4 profile, push-ins with  
light change. Works: medium close-up,  
frontal, soft flat light, approved still as  
first reference.');
```

PART 2 – The production loop

Every video, without exception, runs this loop.

1. Brief → scenes

Give Claude the idea. It writes the scene list and inserts rows into `scenes` with status `draft`. No generation yet.

2. Rung 1 – the still

For every scene with a face in it, generate a still first (Nano Banana Pro, your reference photos attached, *do not alter the face*).

Approve or reject. Approved stills go to `assets` with `approved = true`, `is_start_frame = true`.

Never skip this rung on a face shot. This is the single biggest cost saver you have – a rejected still costs cents, a rejected final take costs dollars.

3. Rung 2 – the motion test

Feed the approved still as the start frame. Veo 3.1 Lite, 720p, 5 seconds, no audio. ~\$0.03/sec → about **\$0.15 per test**.

Judge only two things: does the movement read, and are there artifacts.

Ignore resolution and colour – those come at rung 3.

Log every attempt in `generations` with `rung = 2`.

Failed attempts too –

the failure log is what teaches you which prompts waste money.

4. Rung 3 — the final take

Only for scenes marked `approved`. State the cost first (`duration × rate`), get approval, then run once.

5. Voice and assembly

ElevenLabs for the voiceover. Then ffmpeg locally:

```
# stitch approved clips
ffmpeg -f concat -safe 0 -i list.txt -c
copy stitched.mp4

# lay the voiceover over it
ffmpeg -i stitched.mp4 -i voice.mp3 -c:v
copy -c:a aac -shortest final.mp4
```

No GPU required — this is CPU work.

6. Harvest the template

This step is what makes the project a project. Any scene that worked in three iterations or fewer gets saved to `templates` with its variable parts replaced by `{{placeholders}}`:

```
insert into templates (name, category,
prompt_skeleton, motion, times_used)
values ('close-up reveal', 'reveal',
      '{{CHARACTER_LOCK}} in
{{LOCATION}}', medium close-up frontal
```

```
framing, ...',  
        'slow push in, 15% over 5 seconds',  
1);
```

Next time you need that shot type, you start from the template, not from nothing.

PART 3 — Watching the economics

Run this after every video:

```
select * from project_economics;
```

The number that matters is `cost_per_scene`. It should fall from video to video. If it stays flat, your template library isn't being used — that is the signal to stop producing and spend a session consolidating templates instead.

Track total generations per approved scene too:

```
select scene_number, title,  
       total_generations, total_cost_usd  
from scene_economics  
where project_id = '<id>'  
order by total_cost_usd desc;
```

The most expensive scene in each project tells you exactly which shot type needs a better template.

PART 4 — Guardrails

- **Never batch rung-3 calls.** One at a time, each approved separately.
- **Never let a video model invent a face.** Start frame, always.
- **Stop at 80% of budget** and re-plan rather than pushing through.
- **Log failures.** An unlogged failed generation is money spent with nothing learned from it.
- **Cheap generation is not the bottleneck.** Once the pipeline runs, the limiting factor is idea quality, not cost per clip. Spend the saved money and time on scripts.